

7.5 SUMMARY

One of the most important and complex tasks of an operating system is memory management. Memory management involves treating main memory as a resource to be allocated to and shared among a number of active processes. To use the processor and the I/O facilities efficiently, it is desirable to maintain as many processes in main memory as possible. In addition, it is desirable to free programmers from size restrictions in program development.

The basic tools of memory management are paging and segmentation. With paging, each process is divided into relatively small, fixed-size pages. Segmentation provides for the use of pieces of varying size. It is also possible to combine segmentation and paging in a single memory management scheme.

7.6 RECOMMENDED READING AND ANIMATIONS

A thorough discussion of partitioning strategies is found in [KNUT97]. [PRES72] is a classic treatment of linkers and loaders, still well worth reading. [BEAZ01] provides a useful introduction to linkers, shared libraries, and dynamically loadable extension modules. A practical discussion of linking and loading, with numerous OS examples, is [LEVI00].

BEAZ01 Beazley, D.; Ward, B.; and Cooke, I. “The Inside Story on Shared Libraries and Dynamic Loading.” *Computing in Science & Engineering*, September/October 2001.

KNUT97 Knuth, D. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Reading, MA: Addison-Wesley, 1997.

LEVI00 Levine, J. *Linkers and Loaders*. San Francisco: Morgan Kaufmann, 2000.

PRES72 Presser, L., and White, J. “Linkers and Loaders.” *Computing Surveys*, September 1972.



Animations

A set of animation that illustrates concepts from this chapter is available at the Premium Web site. The reader is encouraged to view the animations to reinforce concepts from this chapter.

7.7 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

absolute loading	compaction	dynamic partitioning
buddy system	dynamic linking	dynamic run-time loading

external fragmentation fixed partitioning frame internal fragmentation linkage editor linking loading logical address	logical organization memory management page page table paging partitioning physical address physical organization	protection relative address relocatable loading relocation segment segmentation sharing
--	--	---

Review Questions

- 7.1. What requirements is memory management intended to satisfy?
- 7.2. Why is the capability to relocate processes desirable?
- 7.3. Why is it not possible to enforce memory protection at compile time?
- 7.4. What are some reasons to allow two or more processes to all have access to a particular region of memory?
- 7.5. In a fixed-partitioning scheme, what are the advantages of using unequal-size partitions?
- 7.6. What is the difference between internal and external fragmentation?
- 7.7. What are the distinctions among logical, relative, and physical addresses?
- 7.8. What is the difference between a page and a frame?
- 7.9. What is the difference between a page and a segment?

Problems

- 7.1. In Section 2.3, we listed five objectives of memory management, and in Section 7.1, we listed five requirements. Argue that each list encompasses all of the concerns addressed in the other.
- 7.2. Consider a fixed partitioning scheme with equal-size partitions of 2^{16} bytes and a total main memory size of 2^{24} bytes. A process table is maintained that includes a pointer to a partition for each resident process. How many bits are required for the pointer?
- 7.3. Consider a dynamic partitioning scheme. Show that, on average, the memory contains half as many holes as segments.
- 7.4. To implement the various placement algorithms discussed for dynamic partitioning (Section 7.2), a list of the free blocks of memory must be kept. For each of the three methods discussed (best-fit, first-fit, next-fit), what is the average length of the search?
- 7.5. Another placement algorithm for dynamic partitioning is referred to as worst-fit. In this case, the largest free block of memory is used for bringing in a process.
 - a. Discuss the pros and cons of this method compared to first-, next-, and best-fit.
 - b. What is the average length of the search for worst-fit?
- 7.6. This diagram shows an example of memory configuration under dynamic partitioning, after a number of placement and swapping-out operations have been carried out. Addresses go from left to right; gray areas indicate blocks occupied by processes; white areas indicate free memory blocks. The last process placed is 2-Mbyte and is marked with an X. Only one process was swapped out after that.



- a. What was the maximum size of the swapped-out process?
 - b. What was the size of the free block just before it was partitioned by X?
 - c. A new 3-Mbyte allocation request must be satisfied next. Indicate the intervals of memory where a partition will be created for the new process under the following four placement algorithms: best-fit, first-fit, next-fit, and worst-fit. For each algorithm, draw a horizontal segment under the memory strip and label it clearly.
- 7.7. A 1-Mbyte block of memory is allocated using the buddy system.
- a. Show the results of the following sequence in a figure similar to Figure 7.6: Request 70; Request 35; Request 80; Return A; Request 60; Return B; Return D; Return C.
 - b. Show the binary tree representation following Return B.
- 7.8. Consider a buddy system in which a particular block under the current allocation has an address of 011011110000.
- a. If the block is of size 4, what is the binary address of its buddy?
 - b. If the block is of size 16, what is the binary address of its buddy?
- 7.9. Let $\text{buddy}_k(x)$ = address of the buddy of the block of size 2^k whose address is x . Write a general expression for $\text{buddy}_k(x)$.
- 7.10. The Fibonacci sequence is defined as follows:

$$F_0 = 0, \quad F_1 = 1, \quad F_{n+2} = F_{n+1} + F_n, \quad n \geq 0$$

- a. Could this sequence be used to establish a buddy system?
 - b. What would be the advantage of this system over the binary buddy system described in this chapter?
- 7.11. During the course of execution of a program, the processor will increment the contents of the instruction register (program counter) by one word after each instruction fetch, but will alter the contents of that register if it encounters a branch or call instruction that causes execution to continue elsewhere in the program. Now consider Figure 7.8. There are two alternatives with respect to instruction addresses:
- Maintain a relative address in the instruction register and do the dynamic address translation using the instruction register as input. When a successful branch or call is encountered, the relative address generated by that branch or call is loaded into the instruction register.
 - Maintain an absolute address in the instruction register. When a successful branch or call is encountered, dynamic address translation is employed, with the results stored in the instruction register.
- Which approach is preferable?
- 7.12. Consider a simple paging system with the following parameters: 2^{32} bytes of physical memory; page size of 2^{10} bytes; 2^{16} pages of logical address space.
- a. How many bits are in a logical address?
 - b. How many bytes in a frame?
 - c. How many bits in the physical address specify the frame?
 - d. How many entries in the page table?
 - e. How many bits in each page table entry? Assume each page table entry contains a valid/invalid bit.
- 7.13. Write the binary translation of the logical address 0001010010111010 under the following hypothetical memory management schemes, and explain your answer:
- a. a paging system with a 256-address page size, using a page table in which the frame number happens to be four times smaller than the page number
 - b. a segmentation system with a 1K-address maximum segment size, using a segment table in which bases happen to be regularly placed at real addresses: $22 + 4,096 + \text{segment \#}$

- 7.14.** Consider a simple segmentation system that has the following segment table:

Starting Address	Length (bytes)
660	248
1,752	422
222	198
996	604

For each of the following logical addresses, determine the physical address or indicate if a segment fault occurs:

- a. 0, 198
 - b. 2, 156
 - c. 1, 530
 - d. 3, 444
 - e. 0, 222
- 7.15.** Consider a memory in which contiguous segments $S_1, S_2, \dots, S_n$ are placed in their order of creation from one end of the store to the other, as suggested by the following figure:



When segment S_{n+1} is being created, it is placed immediately after segment S_n even though some of the segments $S_1, S_2, \dots, S_n$ may already have been deleted. When the boundary between segments (in use or deleted) and the hole reaches the other end of the memory, the segments in use are compacted.

- a. Show that the fraction of time F spent on compacting obeys the following inequality:

$$F \geq \frac{1-f}{1+kf} \quad \text{where} \quad k = \frac{t}{2s} - 1$$

where

s = average length of a segment, in words

t = average lifetime of a segment, in memory references

f = fraction of the memory that is unused under equilibrium conditions

Hint: Find the average speed at which the boundary crosses the memory and assume that the copying of a single word requires at least two memory references.

- b. Find F for $f = 0.2$, $t = 1,000$, and $s = 50$.

APPENDIX 7A LOADING AND LINKING

The first step in the creation of an active process is to load a program into main memory and create a process image (Figure 7.13). Figure 7.14 depicts a scenario typical for most systems. The application consists of a number of compiled or assembled modules in object-code form. These are linked to resolve any references between modules. At the same time, references to library routines are resolved. The library